UNIVERSIDADE DA CORUÑA

epef

Introducción al Aprendizaje Automático

CURSO 2025/26

MERMORIA PRÁCTICA 1

Máster en Ingeniería Industrial

ALUMNO **Martín Castelo Guillermo**

FECHA **Marzo 2025**

ÍNDICE

1. Introducción 3
2. Análisis del dataset 3
3. Preparación de los datos 4
3.1 Separación de variables 4
3.2 Eliminación de columnas 4
3.3 Variable categórica 4
3.4 División de datos 4
3.5 Normalización 5
4. Modelo KNN 5
4.1 Idea del modelo 5
4.2 Entrenamiento y predicción 5
4.3 Resultados 5
5. Árbol de decisión 5
5.1 Idea del modelo 5
5.2 Cómo aprende 6
5.3 Entrenamiento y predicción 6
5.4 Resultados 6
6. Comparación de modelos 6
7. Conclusiones 7

1. Introducción

El objetivo de esta práctica es aplicar modelos de clasificación sobre un dataset real para predecir si una máquina falla o no en función de distintas variables.

Para ello se ha seguido un proceso típico:

1. Entender el dataset
2. Preparar los datos
3. Aplicar modelos
4. Evaluar resultados

Los modelos utilizados han sido:

- KNN
- Árbol de decisión

2. Análisis del dataset

Lo primero que se hizo fue cargar el dataset para ver qué contiene.

El dataset tiene 10.000 registros y varias variables relacionadas con el funcionamiento de una máquina (temperatura, velocidad, torque, desgaste, etc.).

La variable clave es:

Machine failure

- 0 → no hay fallo
- 1 → hay fallo

Al revisar los datos, se vio algo importante:

- 9661 casos sin fallo
- 339 casos con fallo

Esto significa que el dataset está desbalanceado, es decir, hay muchos más casos de no fallo que de fallo.

Esto es importante porque puede hacer que un modelo parezca bueno aunque realmente no detecte bien los fallos.

3. Preparación de los datos

Una vez entendido el dataset, se prepararon los datos paso a paso.

3.1 Separación de variables

Se separaron:

- X → variables de entrada
- Y → variable objetivo (Machine failure)

Esto es necesario porque el modelo tiene que aprender a predecir y a partir de qué datos hacerlo.

3.2 Eliminación de columnas

Se eliminaron:

- UDI
- Product ID

Estas columnas son identificadores, no aportan información útil para predecir fallos.

3.3 Variable categórica

La variable Type tenía valores:

- L
- M
- H

Como los modelos no trabajan con texto, se transformó a números:

- L → 0
- M → 1
- H → 2

Esto permite usarla dentro del modelo.

3.4 División de datos

Se dividieron los datos en:

- 80% entrenamiento
- 20% prueba

Esto se hace para entrenar el modelo con unos datos y comprobarlo con otros que no ha visto.

3.5 Normalización

Se aplicó StandardScaler.

Esto sirve para poner todas las variables en la misma escala.

Es especialmente importante en KNN, ya que este modelo calcula distancias.
Si una variable tiene valores mucho más grandes que otra, influye más en el resultado.

4. Modelo KNN

4.1 Idea del modelo

KNN funciona de forma bastante intuitiva. Cuando llega un dato nuevo, busca los datos más parecidos y decide en función de ellos.

En este caso se usaron: n_neighbors = 7

Es decir, mira los 7 casos más parecidos y decide por mayoría.

4.2 Entrenamiento y predicción

El modelo se entrena con:

knn.fit(X_train, y_train)

Y se hacen predicciones con:

y_pred = knn.predict(X_test)

4.3 Resultados

Se evaluó con:

- Accuracy
- Classification report

El resultado fue: Accuracy ≈ 0.999

Además, el modelo detecta bien los fallos (recall alto).

Esto indica que el modelo funciona muy bien para este dataset.

5. Árbol de decisión

5.1 Idea del modelo

El árbol de decisión funciona mediante reglas.

Va haciendo preguntas sobre los datos, por ejemplo:

- si una variable es mayor que cierto valor
- si no lo es

Y según eso sigue un camino hasta llegar a una decisión final:

- fallo
- no fallo

5.2 Cómo aprende

El modelo analiza los datos y decide automáticamente qué variables usar y en qué orden. Busca dividir los datos de forma que en cada grupo haya casos lo más parecidos posible (fallos por un lado, no fallos por otro).

5.3 Entrenamiento y predicción

Entrenamiento:

tree.fit(X_train, y_train)

Predicción:

y_pred_tree = tree.predict(X_test)

5.4 Resultados

El resultado fue: Accuracy ≈ 0.9975

También detecta bien los fallos, aunque algo peor que KNN en precisión.

6. Comparación de modelos

Comparando ambos modelos:

KNN

- Accuracy más alto
- Mejor precisión en fallos

Árbol de decisión

- Algo menos preciso
- Más fácil de interpretar

En este caso, ambos funcionan muy bien, pero KNN da ligeramente mejores resultados.

7. Conclusiones

Se ha seguido todo el proceso típico de un problema de clasificación:

- análisis del dataset
- preparación de datos
- transformación de variables
- entrenamiento de modelos
- evaluación

El dataset está desbalanceado, lo que hace importante no fijarse solo en el accuracy.

Los dos modelos funcionan bien, pero, KNN es el que mejor resultado da en este caso.
El árbol personalmente me parece más intuitivo pero es algo menos preciso.